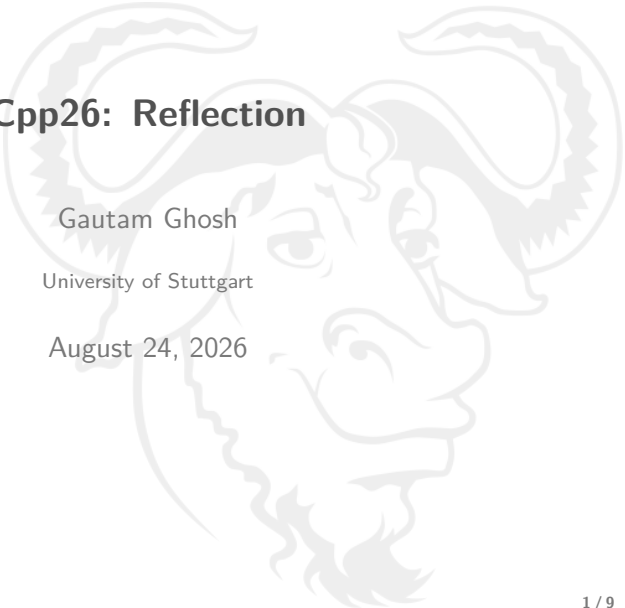


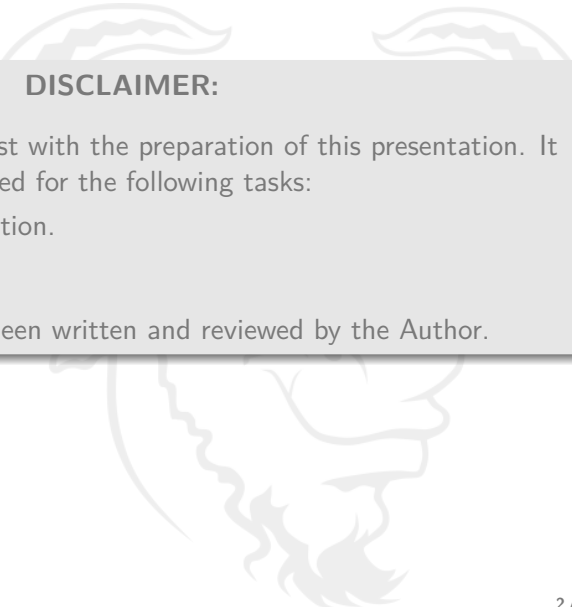


Cpp26: Reflection

Gautam Ghosh

University of Stuttgart

August 24, 2026

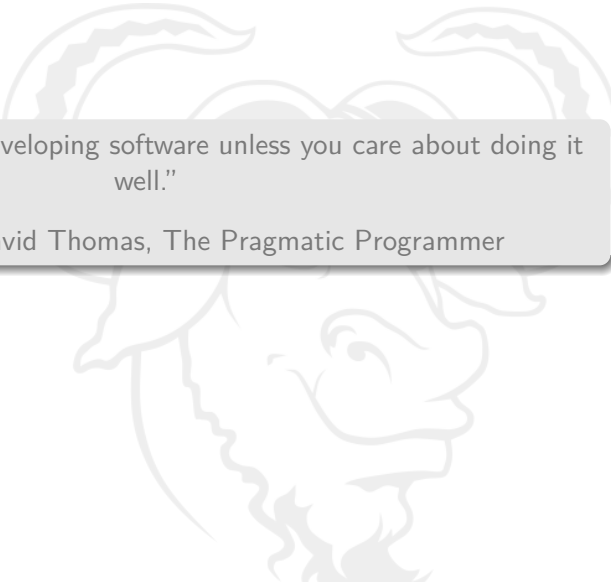


DISCLAIMER:

AI tools were used to assist with the preparation of this presentation. It was used for the following tasks:

- ▶ Navigating documentation.
- ▶ Writing LaTeX code.

All C++ code has been written and reviewed by the Author.



"There is no point in developing software unless you care about doing it well."

Andrew Hunt, David Thomas, The Pragmatic Programmer



C++26

YOUR WISH HAS BEEN GRANTED

The Reflection operator

- ▶ Operator syntax: `^^`
- ▶ Proposed in the paper P2996R13.
- ▶ Encapsulates compile-time information about a C++ entity using the `std::meta::info` type.^[2]

In our initial proposal a value of type `std::meta::info` can represent:

- any (C++) type and type alias
- any function (or member function)
- any variable, static data member, or structured binding
- any non-static data member
- any enumerator
- any template
- any namespace (including the global namespace) or namespace alias
- any object that is a *permitted result of a constant expression*
- any value with *structural type* that is a permitted result of a constant expression
- the null reflection (when default-constructed)

The Splice operator

- ▶ Operator syntax: `[reflection:]`
- ▶ Proposed in the paper P2996R13.
- ▶ Enables converting `std::meta::info` object into **source code**.^[2]
- ▶ Example:

```
constexpr auto r = ^^int;  
typename[:r:] x = 42;           // Same as: int x = 42;  
typename[:^^char:] c = '*';    // Same as: char c = '*';
```

The Expansion Statement

- ▶ Proposed in the paper P1306R5.
- ▶ Enables unrolling for loops at compile time.
- ▶ The compound-statement does not need to be **constexpr**.
- ▶ Example:^[1]

Code	Expands Into
<pre>void f() { template for (constexpr int I : std::array{1, 2, 3}) { static_assert(I < 4); } }</pre>	<pre>void f() { { constexpr auto&& __range = std::array{1, 2, 3}; constexpr auto __begin = __range.begin(); constexpr auto __expansion-size = __range.end() - __begin; // 3 { constexpr int I = *(__begin + 0); static_assert(I < 4); } { constexpr int I = *(__begin + 1); static_assert(I < 4); } { constexpr int I = *(__begin + 2); static_assert(I < 4); } } }</pre>

References

1. Expansion Statements

<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p1306r5.html#expansion-over-tuples>

2. Reflection for C++26

<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p2996r13.html#stdmetainfo>

3. cppreference

<https://en.cppreference.com/cpp/header/meta>

4. C++26 reflection at compile-time

<https://andreasfertig.com/blog/2025/08/cpp26-reflection-at-compile-time/>

Thank You!

Questions?

Code available on GitHub:

